

HIGHER NASH TOOLS

User Manual

Computational tools for higher Nash modifications and singularities

This manual explains how to run the project examples, use the notebook, inspect the main modules, and reproduce the computational computations associated with the thesis.

Contents

1	Project description	2
2	Structure	2
3	Local installation	2
3.1	Download as ZIP	2
3.2	Cloning with Git	3
3.3	Installing dependencies	3
4	Quick verification	3
5	E6 example	3
6	Main modules	4
6.1	Higher Jacobian	4
6.2	Curves	4
6.3	Orders	5
6.4	Matching	5
6.5	Optimal sets	5
6.6	Minors and separation	5
7	Using the notebook	5
7.1	Google Colab	5
7.2	Local use	5
8	Reproducibility and scope	6
9	What a negative result means	6
10	Included tests	6
11	Citation and version	6
12	License and reuse	7

1 Project description

higher-nash-tools is a collection of Python programs that implements the computational steps associated with the study of higher Nash modifications. The procedure follows the mathematical structure of the thesis: construction of the Higher Jacobian, parameterized curves, orders of vanishing, order matrix, minimum-cost matching, enumeration of optimal column sets, maximal minors, and separation using ratios of minors in the Grassmannian. [1]

Scope

The project is intended for research and reproduction of computations. It is not meant to be a general package for all singularities and it does not replace the geometric arguments in the thesis.

The public repository is:

<https://github.com/AlexTenoVaz/higher-nash-tools>

2 Structure

```
higher-nash-tools/  
|-- README.md  
|-- requirements.txt  
|-- source/  
|   |-- curves.py  
|   |-- exponents.py  
|   |-- grassmannian.py  
|   |-- higher_jacobian_matrix.py  
|   |-- matching.py  
|   |-- minors.py  
|   |-- optimal_columns.py  
|   |-- order.py  
|   |-- order_matrix.py  
|   |-- pipeline.py  
|   |-- weighted_order.py  
|-- tests/  
|-- examples/  
|   |-- e6_pipeline.py  
|-- notebooks/  
|   |-- Higher_Jacobian_Matrix.ipynb
```

The dependencies declared in `requirements.txt` are SymPy, NetworkX, and pytest. [2]

3 Local installation

There are two simple ways to get the project.

3.1 Download as ZIP

On GitHub, select **Code** and then **Download ZIP**. Extract the file and open a terminal in the root folder of higher-nash-tools.

3.2 Cloning with Git

If you use Git, you can download the repository with:

```
git clone https://github.com/AlexTenoVaz/higher-nash-tools.git
cd higher-nash-tools
```

In simple terms, “cloning means downloading a copy of the project and keeping it connected to the GitHub repository so it can be updated later.

3.3 Installing dependencies

On Windows:

```
python -m pip install -r requirements.txt
```

4 Quick verification

Before using the software, it is recommended to run the tests:

```
python -m pytest -q
```

If the tests finish successfully, you can run the E6 example:

```
python examples\e6_pipeline.py
```

For the verified project version, the example produces:

```
Leading exponents:
[8, 6, 4]

Order matrix shape:
(35, 55)

Optimal matching cost:
226

Number of optimal column sets:
66
```

5 E6 example

The example uses

$$f = x^2 + y^3 + z^4, \quad m = 5.$$

From two parameterized curves on the chosen divisor, the program obtains the order vector

$$e = (8, 6, 4).$$

Using this vector, the program builds an order matrix of size

$$35 \times 55.$$

A minimum-cost matching problem is then solved, giving

$$c^* = 226.$$

Finally, all column sets that attain this cost are enumerated; the validated example finds

66

conjuntos.

Important

The matching is a combinatorial tool for locating candidates. The optimal cost does not replace the exact computation of the minors. Symbolic cancellations can increase the actual order of a minor.

6 Main modules

Module	Main function
higher_jacobian_matrix.py	Builds the multi-index sets and the Higher Jacobian.
curves.py	Constructs curves on a chosen divisor and performs the descent through blow-ups.
exponents.py	Extracts the leading orders of the coordinates of a curve.
order.py	Computes the order of vanishing with respect to a parameter.
weighted_order.py	Computes the weighted order of a polynomial.
order_matrix.py	Builds the order matrix of the Higher Jacobian.
matching.py	Solves the minimum-cost matching.
optimal_columns.py	Enumerates the optimal column sets.
minors.py	Computes exact maximal minors.
grassmannian.py	Tests the separation of two curves using ratios of minors.
pipeline.py	Integrates the main computational stages.

6.1 Higher Jacobian

The entry (β, α) of the Higher Jacobian is built using

$$\text{Jac}_{\beta, \alpha}^{(m)}(f) = \begin{cases} \frac{1}{(\alpha - \beta)!} \partial^{\alpha - \beta} f, & \alpha \geq \beta, \\ 0, & \text{otherwise.} \end{cases}$$

The main functions are:

```
1 custom_sort
2 default_row_vec
3 default_col_vec
4 derivative_dictionary
5 jacobian_matrix
```

The implementation uses a SymPy SparseMatrix.

6.2 Curves

curves_on_divisor constructs two curves by symbolically solving the divisor equation using parameterized data. descend_curve applies the blow-up transformations in reverse order. [3]

The current implementation works with exactly three variables. The chart argument uses zero-based indices: 0, 1, 2.

6.3 Orders

leading_exponents produces

$$(e_1, e_2, e_3) = (\text{ord}_t \gamma_1, \text{ord}_t \gamma_2, \text{ord}_t \gamma_3).$$

weighted_order computes

$$w_e(g) = \min_{c_\alpha \neq 0} \langle \alpha, e \rangle.$$

6.4 Matching

minimum_cost_matching models the order matrix as a minimum-cost flow network and uses NetworkX's network_simplex. Values equal to $+\infty$ are treated as forbidden edges; zeros are valid costs. [4]

6.5 Optimal sets

enumerate_optimal_column_sets enumerates solutions at the *column set* level, not at the complete matching level. This distinction is the relevant one for maximal minors. [5]

6.6 Minors and separation

maximal_minor computes the determinant of the submatrix formed by all rows and a column set whose size equals the number of rows.

grassmannian_separation looks for a pair (M_p, M_q) such that

$$\lim_{t \rightarrow 0} \frac{M_q(\gamma_1(t))}{M_p(\gamma_1(t))} \neq \lim_{t \rightarrow 0} \frac{M_q(\gamma_2(t))}{M_p(\gamma_2(t))},$$

with both limits finite.

7 Using the notebook

The notebook

notebooks/Higher_Jacobian_Matrix.ipynb

is a reproducible environment for introducing the implementation and running examples.

7.1 Google Colab

It can be opened from the repository through the GitHub link or directly:

https://colab.research.google.com/github/AlexTenoVaz/higher-nash-tools/blob/main/notebooks/Higher_Jacobian_Matrix.ipynb

In Colab, run the cells from the beginning. The notebook prepares the repository copy and the required dependencies.

7.2 Local use

It can also be opened with Jupyter in a local Python installation. The notebook detects the local environment and uses the existing repository copy.

Important

The purpose of the notebook is to explain and reproduce. The main software remains in source/.

8 Reproducibility and scope

The project distinguishes between two different tasks:

1. reduce the combinatorial search using orders and matching;
2. verify the minors exactly and apply the separation criterion.

The first can be relatively fast even for large matrices. The second may require expensive symbolic determinants. For this reason, the personal website presents reproducible computations for the stages that are already automated without trying to turn the browser into a general symbolic computation system.

The thesis also notes that computational separation for selected curves does not replace the geometric argument required to conclude a result about an exceptional component. [1]

9 What a negative result means

A result

separated=False

does not mean “the curves cannot be separated. It means that, with the minors supplied to the procedure, no pair producing two finite and different limits was found.

Similarly, the order matrix provides a combinatorial bound for the behavior of the minors, but it does not by itself determine the exact order when cancellations may occur. [1]

10 Included tests

The repository contains tests for:

- Higher Jacobian;
- curves and descent;
- orders and weighted orders;
- order matrix;
- matching;
- enumeration of optimal sets;
- minors;
- separation in the Grassmannian;
- integrated E6 and A2 pipelines.

In particular, the regression tests fix the behavior of the E6 example, including the cost 226 and the 66 optimal column sets. [6]

11 Citation and version

The repository can be cited as research software. For a version specifically associated with a thesis, it is useful to identify the version or commit used to generate the results.

A simple reference is:

José Alejandro Tenorio Vázquez, *higher-nash-tools: Computational tools for the study of higher Nash modifications and singularities*, GitHub, 2026.

12 License and reuse

The current repository is public. The version consulted in September 2026 does not declare a software license in the repository metadata. Therefore, reuse of the code should be considered with caution until the author chooses an explicit license.

References

- [1] J. A. Tenorio Vázquez, *Master's Thesis*, Instituto de Matemáticas, Universidad Nacional Autónoma de México, 2026.
- [2] A. Tenorio Vázquez, *higher-nash-tools: requirements.txt*, 2026.
<https://github.com/AlexTenoVaz/higher-nash-tools/blob/main/requirements.txt>
- [3] A. Tenorio Vázquez, *higher-nash-tools: source/curves.py*, 2026.
<https://github.com/AlexTenoVaz/higher-nash-tools/blob/main/source/curves.py>
- [4] A. Tenorio Vázquez, *higher-nash-tools: source/matching.py*, 2026.
<https://github.com/AlexTenoVaz/higher-nash-tools/blob/main/source/matching.py>
- [5] A. Tenorio Vázquez, *higher-nash-tools: source/optimal_columns.py*, 2026.
https://github.com/AlexTenoVaz/higher-nash-tools/blob/main/source/optimal_columns.py
- [6] A. Tenorio Vázquez, *higher-nash-tools: tests/test_optimal_columns.py*, 2026.
https://github.com/AlexTenoVaz/higher-nash-tools/blob/main/tests/test_optimal_columns.py